SQL Practice — Orders Table

WHERE, HAVING, GROUP BY & ORDER BY — CONCISE VERSION

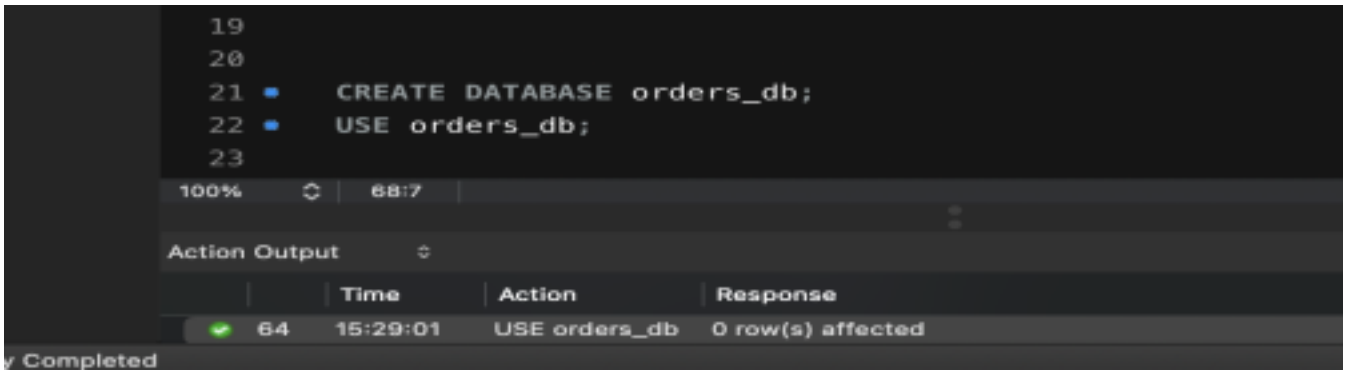
Name: Vaishali

Roll No: 2520030117

Section: 7

Step 1 — Create Database

```
CREATE DATABASE orders_db;
USE orders_db;
```



Step 2 — Create Orders Table (DDL)

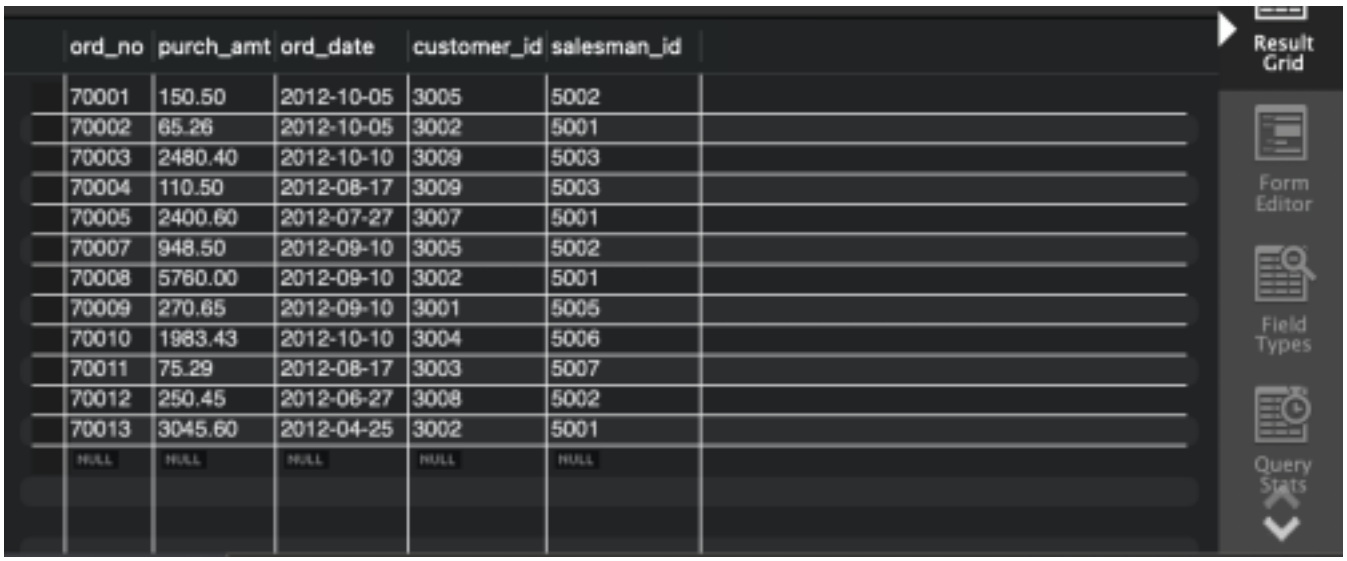
```
CREATE TABLE orders (
    ord_no INT PRIMARY KEY,
    purch_amt DECIMAL(10,2),
    ord_date DATE,
    customer_id INT,
    salesman_id INT
);
```



Step 3 — Insert Data into Orders and Display

```
INSERT INTO orders VALUES
(70001,150.50,'2012-10-05',3005,5002),
(70009,270.65,'2012-09-10',3001,5005),
(70002,65.26,'2012-10-05',3002,5001),
(70004,110.50,'2012-08-17',3009,5003),
(70007,948.50,'2012-09-10',3005,5002),
(70005,2400.60,'2012-07-27',3007,5001),
(70008,5760.00,'2012-09-10',3002,5001),
(70010,1983.43,'2012-10-10',3004,5006),
(70003,2480.40,'2012-10-10',3009,5003),
(70012,250.45,'2012-06-27',3008,5002),
(70011,75.29,'2012-08-17',3003,5007),
(70013,3045.60,'2012-04-25',3002,5001);

SELECT * FROM orders;
```



ord_no	purch_amt	ord_date	customer_id	salesman_id
70001	150.50	2012-10-05	3005	5002
70002	65.26	2012-10-05	3002	5001
70003	2480.40	2012-10-10	3009	5003
70004	110.50	2012-08-17	3009	5003
70005	2400.60	2012-07-27	3007	5001
70007	948.50	2012-09-10	3005	5002
70008	5760.00	2012-09-10	3002	5001
70009	270.65	2012-09-10	3001	5005
70010	1983.43	2012-10-10	3004	5006
70011	75.29	2012-08-17	3003	5007
70012	250.45	2012-06-27	3008	5002
70013	3045.60	2012-04-25	3002	5001
NULL	NULL	NULL	NULL	NULL

Result Grid
Form Editor
Field Types
Query Stats

Step 4 — Question 1: WHERE Clause

```
SELECT *
FROM orders
WHERE purch_amt > 2000;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
NULL	NULL	NULL	NULL	NULL

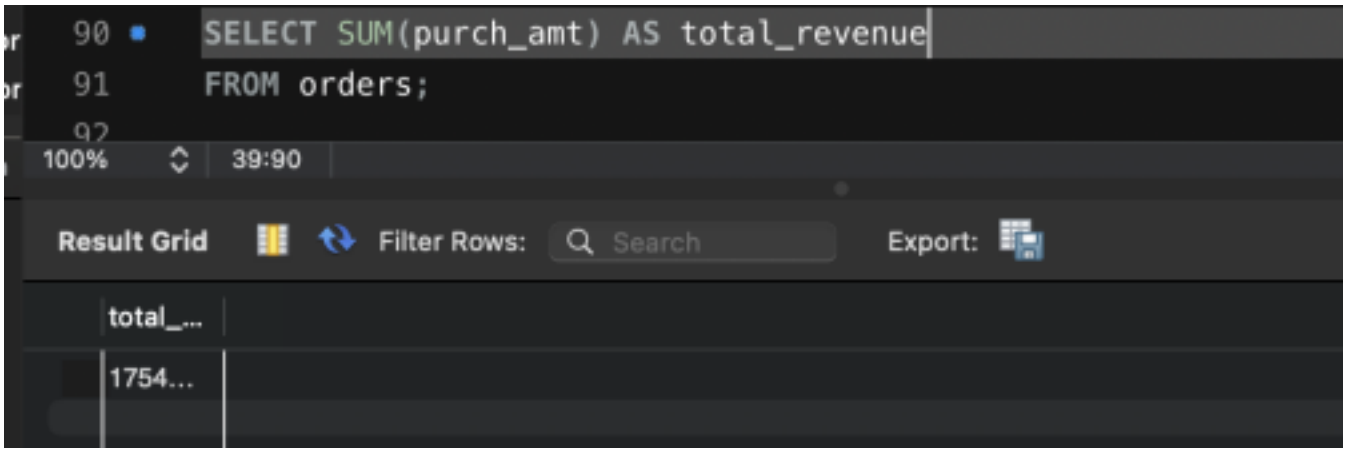
Step 5 — Question 2: ORDER BY

```
SELECT *
FROM orders
ORDER BY purch_amt DESC;
```

ord_no	purch_amt	ord_date	customer_id	salesman_id
70008	5760.00	2012-09-10	3002	5001
70013	3045.60	2012-04-25	3002	5001
70003	2480.40	2012-10-10	3009	5003
70005	2400.60	2012-07-27	3007	5001
70010	1983.43	2012-10-10	3004	5006
70007	948.50	2012-09-10	3005	5002
70009	270.65	2012-09-10	3001	5005
70012	250.45	2012-06-27	3008	5002
70001	150.50	2012-10-05	3005	5002
70004	110.50	2012-08-17	3009	5003
70011	75.29	2012-08-17	3003	5007
70002	65.26	2012-10-05	3002	5001
NULL	NULL	NULL	NULL	NULL

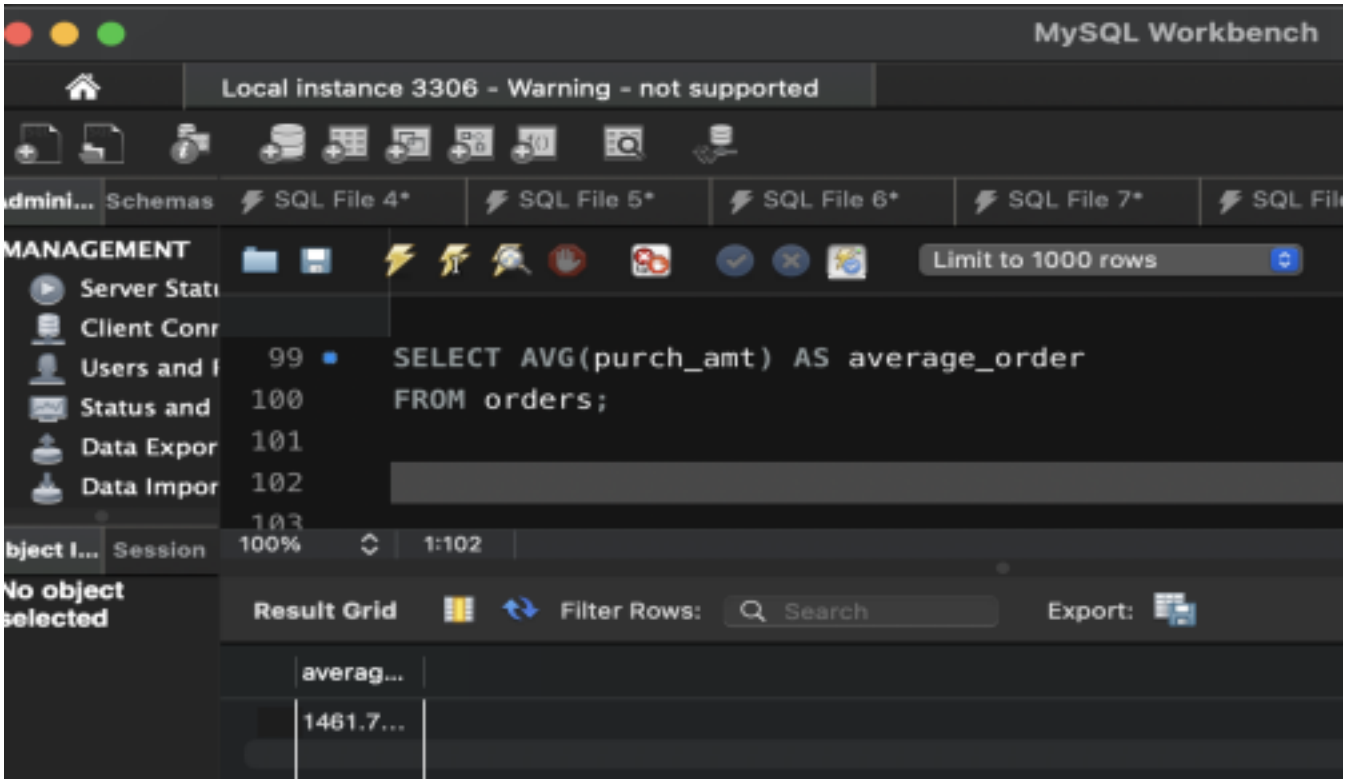
Step 6 — Question 3: SUM()

```
SELECT SUM(purch_amt) AS total_revenue
FROM orders;
```



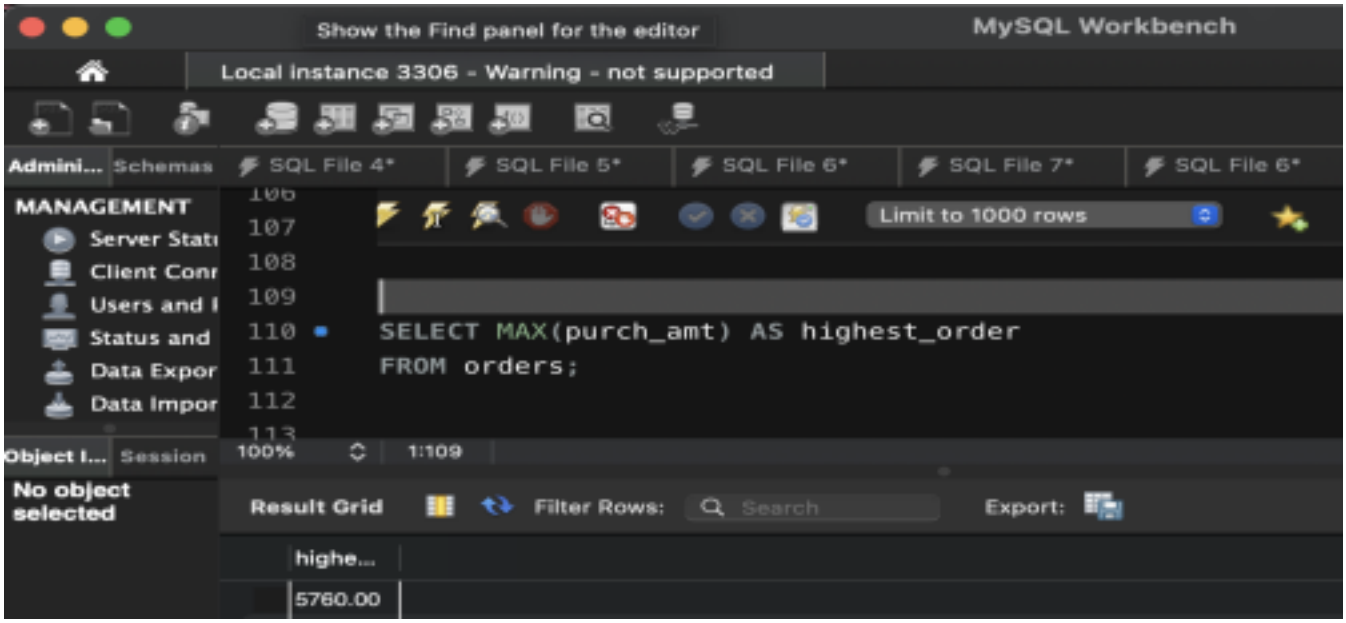
Step 7 — Question 4: AVG()

```
SELECT AVG(purch_amt) AS average_order
FROM orders;
```



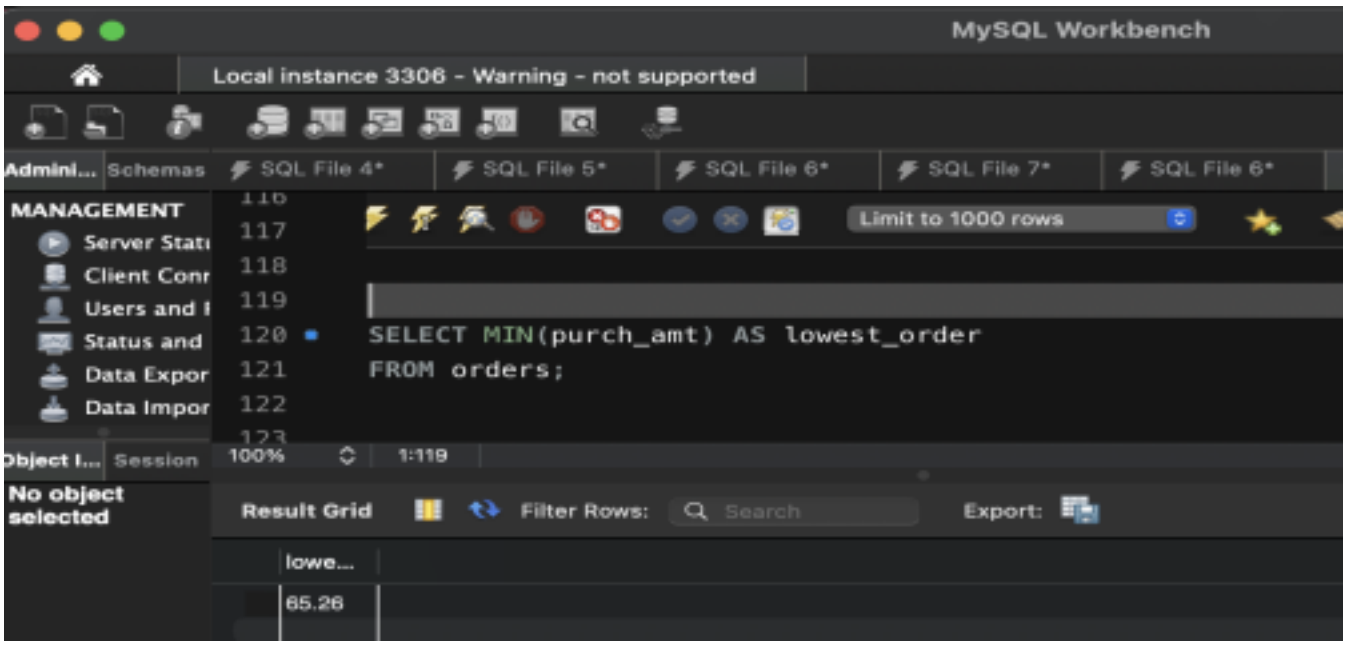
Step 8 — Question 5: MAX()

```
SELECT MAX(purch_amt) AS highest_order
FROM orders;
```



Step 9 — Question 6: MIN()

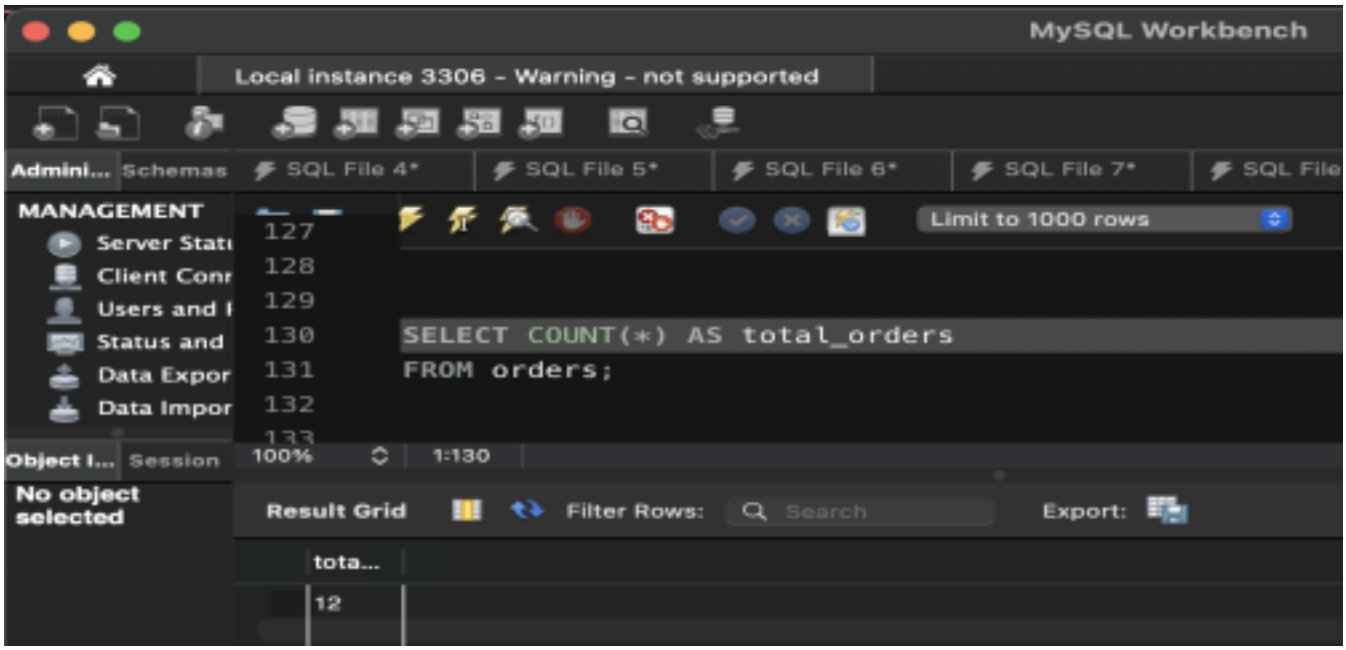
```
SELECT MIN(purch_amt) AS lowest_order
FROM orders;
```



Step 10 — Question 7: COUNT()

Scenario: Find total number of orders.

```
SELECT COUNT(*) AS total_orders
FROM orders;
```



Step 11 — Question 8: GROUP BY

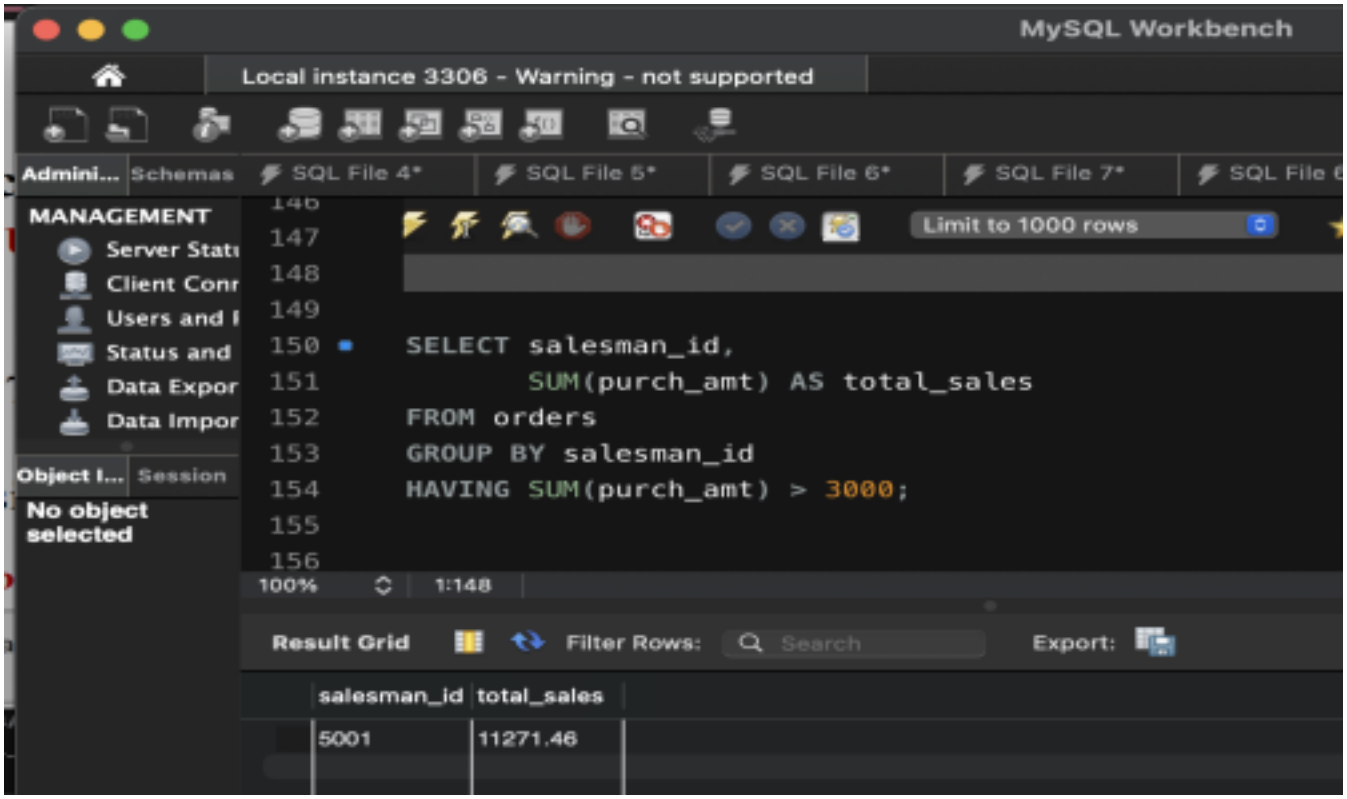
```
SELECT salesman_id,
       SUM(purch_amt) AS total_sales
FROM orders
GROUP BY salesman_id;
```

salesman_id	total_sales
5002	1349.45
5001	11271.46
5003	2590.90
5005	270.65
5006	1983.43
5007	75.29

Step 12 — Question 9: GROUP BY + HAVING

Scenario: Company rewards salesmen whose total sales exceed $3,000.

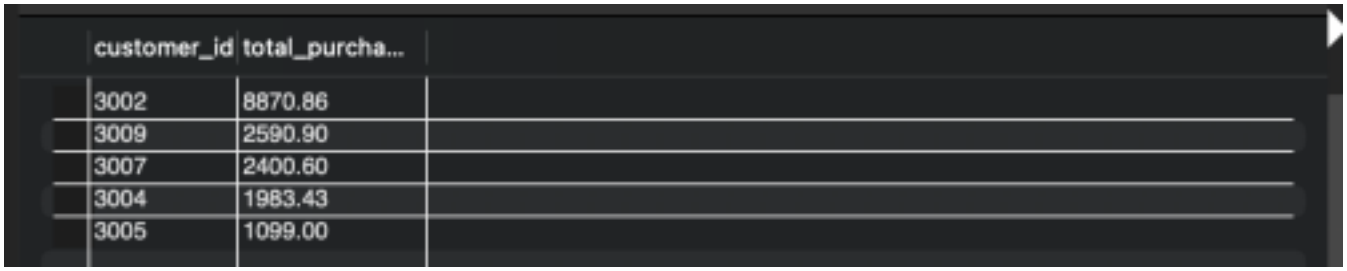
```
SELECT salesman_id,
       SUM(purch_amt) AS total_sales
FROM orders
GROUP BY salesman_id
HAVING SUM(purch_amt) > 3000;
```



Step 13 — Question 10: GROUP BY + HAVING + ORDER BY

Scenario: Management wants top customers ranked by spending.

```
SELECT customer_id,
       SUM(purch_amt) AS total_purchase
FROM orders
GROUP BY customer_id
HAVING SUM(purch_amt) > 1000
ORDER BY total_purchase DESC;
```



customer_id	total_purcha...
3002	8870.86
3009	2590.90
3007	2400.60
3004	1983.43
3005	1099.00

Step 14 — Question 11: BETWEEN + HAVING

Scenario: Find customers whose maximum purchase is between $2,000 and $6,000.

```
SELECT customer_id,
       MAX(purch_amt) AS max_purchase
FROM orders
GROUP BY customer_id
HAVING MAX(purch_amt) BETWEEN 2000 AND 6000;
```



customer_id	max_purchase
3002	5760.00
3009	2480.40
3007	2400.60

Step 15 — Question 12: MAX + GROUP BY + HAVING

Scenario: Management wants daily highest purchases above $2,000.

```
SELECT ord_date,
       MAX(purch_amt) AS highest_purchase
FROM orders
GROUP BY ord_date
HAVING MAX(purch_amt) > 2000;
```



2012-10-10	2480.40
2012-07-27	2400.60
2012-09-10	5760.00
2012-04-25	3045.60